

OmniAI Gateway

Proposta de Projeto

Guilherme Coutinho (Nº 50467)
A50467@alunos.isel.pt
916 808 834

André Nunes (Nº 51766)
A51766@alunos.isel.pt
915 124 399

Orientador: Pedro Félix (pedro.felix@isel.pt)

Data: 9 de Março de 2026

1 Contexto

Os Modelos de Linguagem de Grande Escala (*Large Language Models – LLMs*) [1] tornaram-se componentes relevantes no desenvolvimento de aplicações baseadas em inteligência artificial. Um LLM é um modelo treinado com grandes volumes de texto capaz de entender e gerar linguagem natural e outros tipos de conteúdo, possibilitando a realização de diversas tarefas.

É possível distinguir duas fases principais no ciclo de vida de um modelo, treino e inferência. O treino corresponde ao processo de aprendizagem a partir de grandes conjuntos de dados, exigindo elevados recursos computacionais. A inferência corresponde à utilização do modelo já treinado para gerar respostas a novos pedidos. Atualmente, a maioria das aplicações utiliza modelos exclusivamente em modo de inferência.

O acesso aos LLMs é realizado através das suas APIs de inferência disponibilizadas pelos seus fornecedores. A comunicação com estas APIs é realizada através do protocolo HTTP, utilizando normalmente o formato JSON, tanto na estrutura dos pedidos enviados como nas respostas retornadas.

Estas APIs de inferência convertem internamente um pedido em linguagem natural (*prompt*) para o formato utilizado pelos LLMs, denominado *tokens*. Os *tokens* [2] são pequenas unidades de dados que resultam da divisão de blocos maiores de informação, permitindo ao modelo processar e interpretar os dados de forma eficiente. Como os modelos geram as suas respostas sequencialmente em forma de múltiplos *tokens*, muitas APIs de inferência suportam mecanismos de *streaming*, permitindo minimizar a latência sentida no seu uso.

Atualmente, múltiplos fornecedores disponibilizam serviços de inferência, incluindo OpenAI [3], Anthropic [4], Google [5] e Groq [6]. Apesar de utilizarem JSON na comunicação cada fornecedor define o seu próprio formato de pedido e resposta, bem como diferentes mecanismos de autenticação. A integração de vários fornecedores requer assim a adaptação a diferentes interfaces de comunicação e estruturas de dados.

O uso destes modelos evoluiu de simples interações para sistemas baseados em agentes (*Agentic Workflows*) [7]. Nestes cenários, os LLMs são capazes de delegar tarefas, interpretar resultados e tomar decisões intermédias de forma autónoma.

Com o crescimento destes sistemas baseados em agentes, surgiu a necessidade de criar um mecanismo padrão para a comunicação entre modelos e ferramentas externas, nomeadamente o *Model Context Protocol* (MCP) [8], que define uma arquitetura cliente-servidor para facilitar a troca de informações entre modelos e ferramentas de forma estruturada.

2 Problema

A existência de múltiplos fornecedores de LLMs e a ausência de um protocolo comum implica que existam diferentes interfaces de acesso aos modelos, com estruturas específicas de pedidos e respostas, modelos distintos de contabilização de *tokens*, custos variáveis e implementações particulares para invocação de funções externas (*tool calling*). Por exemplo, enquanto a API de um fornecedor pode exigir a definição de ferramentas num esquema JSON rigoroso, outros adotam sintaxes baseadas em XML ou estruturas proprietárias.

Apesar de existirem mecanismos criados para mitigar estas diferenças, como o *Model Context Protocol* (MCP), o mesmo ainda não é amplamente adotado pelas APIs de inferência.

Estas diferenças tecnológicas criam um forte acoplamento entre as aplicações cliente e as APIs dos fornecedores. Sempre que uma aplicação pretende integrar mais do que um modelo de linguagem ou ferramenta, torna-se necessário implementar lógica específica para cada API, incluindo:

- Conversão de formatos de pedido e resposta;
- Interpretação de estruturas de dados e respostas específicas;
- Diferente gestão de erros e códigos de estado HTTP;
- Implementação manual de mecanismos de redundância ou *fallback*.

Como consequência deste acoplamento tecnológico, observam-se nas aplicações modernas [9] [10]:

- Duplicação de código e aumento da complexidade;
- Dificuldade na substituição ou combinação dinâmica de diversos modelos;
- Baixa portabilidade das aplicações entre diferentes fornecedores;
- Aumento significativo do esforço de manutenção a longo prazo;
- Maior exposição à dependência tecnológica (*vendor lock-in*) de um único fornecedor.

Outro fator crítico está relacionado com o tratamento de dados sensíveis e a segurança da infraestrutura. A execução de ferramentas localmente ou o envio de contexto interno para modelos externos de terceiros levanta questões de privacidade. Além disso, a falta de uma camada intermédia estruturada dificulta a aplicação de mecanismos de filtragem, controlo de acesso e auditoria.

Neste contexto, o principal problema é a ausência de uma camada de abstração capaz de separar as aplicações cliente das particularidades de cada fornecedor de LLMs, garantindo simultaneamente a execução segura das ferramentas externas.

3 Solução

A figura 1 apresenta a arquitetura do sistema idealizado a realizar:

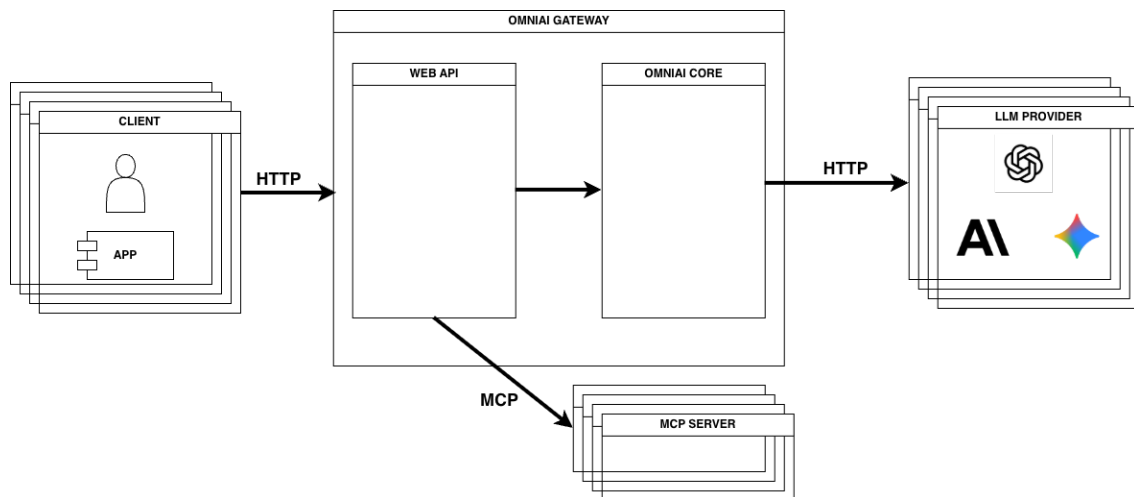


Figura 1: Diagrama da Arquitetura do Sistema

A solução proposta, denominada **OmniAI Gateway**, consiste no desenvolvimento de um sistema informático que atua como uma *Gateway*, ou seja, um ponto de entrada único que centraliza o acesso a diversas APIs. Ao orientar esta centralização para APIs de inferência e de ferramentas via MCP, o sistema assume-se como uma *AI Gateway*. Este sistema informático procura resolver o problema das diferenças tecnológicas no acesso a LLMs e a servidores MCP. Embora partilhe características com sistemas existentes, como o *LiteLLM* [11] ou o *OpenRouter* [12], o OmniAI Gateway distingue-se pela centralização e execução de ferramentas a pedido da *Gateway*.

Para a realização do sistema, a arquitetura será dividida em duas componentes principais. Primeiramente, será desenvolvida uma biblioteca base em Kotlin, designada OmniAI Core. Esta biblioteca estabelecerá uma camada de abstração implementando um *pipeline* de processamento que garante as seguintes etapas:

- **Acesso a APIs de Inferência:** Integração com diversas APIs de modelos de IA, gerindo a comunicação e a autenticação em cada fornecedor.
- **Roteamento Inteligente de Modelos:** Análise do pedido para otimização de custos e garantia de fiabilidade através de mecanismos de *fallback* e *retry* automático em caso de falha, latência elevada ou indisponibilidade de um fornecedor primário.
- **Invocação e Registo de Ferramentas (via MCP):** Disponibilização de uma interface para o registo dinâmico de ferramentas externas. Isto permite abstrair a complexidade da execução para a aplicação cliente, gerindo o ciclo de *tool calling* [13], conforme apresentado na Figura 2.
- **Normalização e Monitorização:** Normalização dos formatos de pedido, resposta e erros entre pelo menos dois ecossistemas de fornecedores distintos, bem como a recolha de métricas de utilização, contagem de *tokens*, monitorização de latência e controlo de custos operacionais.

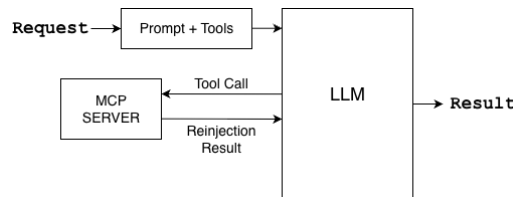


Figura 2: Representação do ciclo de *tool calling* e interação com ferramentas externas.

Após a consolidação da biblioteca Core, será desenvolvida a aplicação final, denominada OmniAI Gateway. Esta camada superior irá expor uma API HTTP, com suporte a mecanismos de *streaming* ou comunicação assíncrona, de forma a reduzir a latência percebida pelos utilizadores.

O objetivo desta aplicação é servir como interface de comunicação e demonstrar um cenário de utilização real da biblioteca desenvolvida. A implementação incluirá alguns fornecedores de LLMs e ferramentas MCP, suficientes para demonstrar o funcionamento completo da arquitetura proposta, sem a necessidade de suportar todos os fornecedores existentes.

4 Riscos e Mitigações

Durante a fase de planeamento do projeto, foram identificados os seguintes riscos que podem impactar o cronograma e a qualidade dos resultados, bem como as respetivas estratégias de mitigação:

- **Curva de Aprendizagem Tecnológica:** A utilização de protocolos recentes (como o MCP) pode provocar alguns atrasos nas fases iniciais do desenvolvimento.
Mitigação: Realização de investigação nas primeiras semanas, com foco na criação de pequenos protótipos utilizando MCP antes da sua integração na arquitetura final.
- **Atrasos no Planeamento e Integração:** A complexidade associada à normalização de múltiplos fornecedores com APIs distintas pode levar a desvios no cronograma inicialmente definido.
Mitigação: Adoção de uma abordagem faseada, iniciando com a integração de dois fornecedores estáveis (ex.: Groq e Gemini).
- **Instabilidade de Serviços Externos:** A dependência direta de APIs de terceiros introduz o risco de indisponibilidade temporária ou limitações de *rate-limiting* durante o desenvolvimento.
Mitigação: Implementação de mecanismos de *mocking* e testes unitários locais no Core, reduzindo a necessidade de chamadas constantes a serviços externos.
- **Custos Imprevistos de Tokens:** Durante a fase de desenvolvimento, funcionalidades como *retries* em *loop* podem gerar um consumo elevado de créditos nas APIs.
Mitigação: Definição de tetos de consumo nas contas dos fornecedores e utilização temporária de modelos locais mais pequenos para testes estruturais.
- **Custo das APIs de Inferência:** Os custos associados às APIs de inferência podem representar uma dificuldade na sua integração no *gateway*, especialmente quando se pretende realizar testes frequentes ou experimentar diferentes fornecedores.
Mitigação: Utilização de créditos de estudante ou créditos iniciais disponibilizados por alguns fornecedores. Adicionalmente, será dada preferência a APIs que ofereçam limites gratuitos razoáveis, permitindo realizar testes e validações sem custos significativos.

5 Planeamento

Semana	Período	Descrição
1	18 fev – 22 fev (parcial)	Estudo das APIs de inferência e protocolo MCP.
2	23 fev – 1 mar	Continuação da semana anterior e início da proposta.
3	2 mar – 8 mar	Entrega Proposta do projeto
4	9 mar – 15 mar	Configuração inicial da biblioteca, definição da arquitetura base e das interfaces.
5	16 mar – 22 mar	Integração com as APIs de inferência. Foco inicial em dois fornecedores estáveis.
6	23 mar – 29 mar	Integração do MCP no OmniAI Core. Invocação e registo dinâmico de ferramentas, gerindo o ciclo de <i>tool calling</i> .
7	30 mar – 5 abr	Implementação do roteamento inteligente de modelos (otimização de custos) e mecanismos de redundância (<i>fallback</i> e <i>retry</i>).
8	6 abr – 12 abr	Implementação dos mecanismos de segurança, como controlo de acesso, preparação do contexto e remoção de dados sensíveis.
9	13 abr – 19 abr	Conclusão das implementações anteriores.
10	20 abr – 26 abr	Apresentação de Progresso
11	27 abr – 3 mai	Início do desenvolvimento da aplicação final OmniAI Gateway. Exposição da WEB API.
12	4 mai – 10 mai	Continuação da semana anterior e integração do MCP na OmniAI Gateway.
13	11 mai – 17 mai	Término de todas as APIs de comunicação para o exterior.
14	18 mai – 24 mai	Testes de integração do Core e da Gateway.
15	25 mai – 31 mai	Entrega da Versão beta
16	1 jun – 7 jun	Refinamento de testes a todos os módulos.
17	8 jun – 14 jun	Refinamento de código, desempenho e limpeza de bugs do sistema.
18	15 jun – 21 jun	Continuação da semana anterior e refinamento do relatório final.
19	22 jun – 28 jun 2026	Conclusão do relatório final.
20	29 jun – 5 jul 2026	Vistoria completa a todo o sistema e polimento de falhas para a entrega da Versão final.
21	6 jul – 11 jul 2026 (parcial)	Entrega Final do Projeto e Relatório

Tabela 1: Planeamento semanal do projeto

Referências

- [1] IBM, “What are large language models (LLMs)?” [Online]. Available: <https://www.ibm.com/think/topics/large-language-models>
- [2] NVIDIA, “Ai tokens explained.” [Online]. Available: <https://blogs.nvidia.com/blog/ai-tokens-explained/>
- [3] OpenAI, “Openai api documentation.” [Online]. Available: <https://platform.openai.com/docs/>
- [4] Anthropic, “Anthropic messages api documentation.” [Online]. Available: <https://docs.anthropic.com/en/api/messages>
- [5] Google, “Gemini api documentation.” [Online]. Available: <https://ai.google.dev/docs>
- [6] Groq, “Groq api documentation.” [Online]. Available: <https://console.groq.com/docs>
- [7] IBM, “What are agentic workflows?” [Online]. Available: <https://www.ibm.com/think/topics/agentic-workflows>
- [8] Anthropic, “Model context protocol specification.” [Online]. Available: <https://modelcontextprotocol.io>
- [9] Requesty, “Switching llm providers: Why it’s harder than it seems,” 2025. [Online]. Available: <https://www.requesty.ai/blog/switching-llm-providers-why-it-s-harder-than-it-seems>
- [10] A. Basu, “The api trap: Why companies risk lock-in with LLM providers,” 2025, medium. [Online]. Available: <https://medium.com/@aritrabasu2001/the-api-trap-why-companies-risk-lock-in-with-llm-providers-6754d98a7661>
- [11] Berri AI, “LiteLLM: Call all llm apis using the openai format.” [Online]. Available: <https://www.litellm.ai/>
- [12] OpenRouter, “OpenRouter: A unified interface for llms.” [Online]. Available: <https://openrouter.ai/>
- [13] DAIR.AI, “Function calling and tool use in LLM agents.” [Online]. Available: <https://www.promptingguide.ai/agents/function-calling>